

Шпаргалка Python

Типы данных

- **int** (целые числа):

- Могут быть положительными или отрицательными.
- Не имеют максимального размера (зависит от памяти).

Примеры:

```
a = 123
b = -456
```

- **float** (числа с плавающей точкой):

- Представляют дробные числа.
- Имеют ограниченную точность из-за внутреннего представления в памяти.

Примеры:

```
c = 3.14
d = -0.001
```

- **complex** (комплексные числа):

- Имеют вещественную и мнимую части.

Пример:

```
e = 2 + 3j
```

- **str** (строки):

- Представляют последовательность символов.
- Иммутабельны (нельзя изменить символ по индексу).

Пример:

```
s = "Привет, мир!"
```

- **list** (списки):

- **tuple** (кортежи):

- Упорядоченная коллекция элементов.
- Могут содержать элементы разных типов.

Примеры:

```
lst1 = [1, 2, 3]
lst2 = ["a", "b", "c"]
```

- Упорядоченная и неизменяемая коллекция элементов.

Пример:

```
tpl = (1, 2, 3)
```

- **set (множества):**
 - Коллекция уникальных и неупорядоченных элементов.

Пример:

```
st = {1, 2, 3}
```

- **dict (словари):**
 - Коллекция пар "ключ-значение".

Пример:

```
dct = {"name": "Анна",
      "age": 25}
```

- **bool (булевы значения):**
 - Имеют два возможных значения: **True** или **False**.

Пример:

```
flag = True
```

- **None :**
 - Отсутствие значения или "ничего".

Пример:

```
n = None
```

Преобразование типов:

- Для преобразования типов данных используются встроенные функции: **int()**, **float()**, **str()**, **list()**, **tuple()**, **set()**, **dict()**.

```
num = 10
str_num = str(num)  # Преобразование числа в строку
```

```
int_str = int("123") # Преобразование строки в число
```

Операторы

1. Арифметические операторы:

- `+` : Сложение
- `-` : Вычитание
- `*` : Умножение
- `/` : Деление (вещественное)
- `//` : Деление (целочисленное)
- `%` : Остаток от деления
- `**` : Возведение в степень

```
result = 5 + 3 # result = 8
result = 5 - 3 # result = 2
result = 5 * 3 # result = 15
result = 5 / 2 # result = 2.5
result = 5 // 2 # result = 2
result = 5 % 2 # result = 1
result = 2 ** 3 # result = 8
```

2. Операторы сравнения:

- `==` : Равно
- `!=` : Не равно
- `<` : Меньше
- `>` : Больше
- `<=` : Меньше или равно
- `>=` : Больше или равно

```
x = 5
y = 10
result = x == y # result = False
result = x != y # result = True
result = x < y # result = True
result = x > y # result = False
result = x <= y # result = True
result = x >= y # result = False
```

3. Логические операторы:

- `and` : И
- `or` : Или

```
x = True
y = False
```

- `not` : Не

```
result = x and y   # result = False
result = x or y    # result = True
result = x not y   # result = True
```

4. Операторы присваивания:

- `=` : Присваивание
- `+=` : Прибавить и присвоить
- `-=` : Вычесть и присвоить
- `*=` : Умножить и присвоить
- `/=` : Поделить и присвоить
- `//=` : Целочисленно поделить и присвоить
- `%=` : Взять остаток и присвоить
- `**=` : Возвести в степень и присвоить

```
x = 5
x += 3   # x = 8
x -= 3   # x = 5
x *= 3   # x = 15
x /= 3   # x = 5
x //= 3  # x = 1
x = 5
x %= 3   # x = 2
x **= 2  # x = 4
```

5. Операторы членства и идентичности:

- `in` : Принадлежит ли элемент контейнеру
- `not in` : Не принадлежит ли элемент контейнеру
- `is` : Ссылаются ли два объекта на одно и то же место в памяти
- `is not` : Не ссылаются ли два объекта на одно и то же место в памяти

```
lst = [1, 2, 3]
result = 1 in lst
# result = True
result = 6 not in lst
# result = True, так как число 6
# не содержится в списке

a = [1, 2, 3]
b = a
# b ссылается на тот же список,
# что и a
result = a is b
```

```
# result = True, так как a и b  
# ссылаются на один и тот же объект  
# в памяти
```

```
a = [1, 2, 3]  
b = [1, 2, 3]  
# b - это новый объект с такими  
# же значениями  
result = a is not b  
# result = True, так как a и b  
# не ссылаются на одно и то же  
# место в памяти
```

Условные операторы

1. Оператор `if` :

Используется для проверки условия. Если условие истинно, выполняется код внутри блока

`if` .

```
x = 10  
if x > 5:  
    print("x больше 5")
```

2. Оператор `if...else` :

Используется для выполнения разных действий в зависимости от истинности условия.

```
x = 4  
if x % 2 == 0:  
    print("x - четное число")  
else:  
    print("x - нечетное число")
```

3. Оператор `if...elif...else` :

Используется, когда нужно проверить несколько условий последовательно.

```
x = 0  
if x > 0:  
    print("x положительное")  
elif x < 0:  
    print("x отрицательное")
```

```
else:  
    print("x равно нулю")
```

4. Вложенные условия:

Условные операторы можно вкладывать друг в друга.

```
x = 10  
y = 5  
if x > 5:  
    if y > 2:  
        print("Оба условия истинны")
```

5. Тернарный оператор:

Позволяет коротко записать условное выражение в одной строке.

```
x = 10  
result = "больше 5" if x > 5 else "не бо  
льше 5"  
print(result)
```

6. Использование логических операторов в условиях:

Можно комбинировать условия с помощью логических операторов (`and` , `or` , `not`).

```
x = 10  
y = 20  
if x > 5 and y > 15:  
    print("Оба условия истинны")
```

Циклы

1. Цикл `for` :

Цикл `for` используется для итерации по коллекции (например, списку, кортежу, строке) или для выполнения кода определенное количество раз.

- **Базовая итерация:**

```
for i in range(5):  
    print(i)  
# Выводит числа от 0 до 4
```

- **Итерация по списку:**

```
lst = [10, 20, 30]  
for num in lst:  
    print(num)
```

```
# Выводит каждое число из списка
```

- **Итерация по строке:**

```
s = "Python"
for char in s:
    print(char)
# Выводит каждую букву строки
```

- **Использование `else` в цикле `for`:**

```
for i in range(3):
    print(i)
else:
    print("Цикл завершен")
# Эта строка выводится после
# завершения цикла
```

2. Цикл `while`:

Цикл `while` выполняет блок кода, пока указанное условие истинно.

- **Базовый пример**

```
count = 0
while count < 5:
    print(count)
    count += 1
# Не забывайте обновлять условие, чтобы избежать бесконечного цикла!
```

- **Прерывание цикла с помощью**

`break`:

```
count = 0
while True:
    if count >= 5:
        break
    # Прерывание цикла при
    # достижении условия
    print(count)
    count += 1
```

- **Пропуск итерации с помощью**

`continue`:

```
count = 0
while count < 5:
    count += 1
```

Использование `else` в цикле `while`:

Также как и с `for`, `while` может иметь блок `else`.

```
count = 0
while count < 5:
```

```
if count == 3:
    continue
    # Пропуск текущей ите-
рации
print(count)
```

```
print(count)
count += 1
else:
    print("Цикл завершен")
# Эта строка выводится
# после завершения цикла
```

Списки

Основные операции

- **Создание списка:**

```
lst = [1, 2, 3, 4, 5]
```

- **Доступ к элементам:**

```
first_element = lst[0]    # 1
last_element = lst[-1]    # 5
```

Методы

1. Основные методы для изменения списка:

- **append(x)** : Добавляет элемент **x** в конец списка.

```
lst.append(6)
```

- **extend(iterable)** : Расширяет список, добавляя элементы из итерируемого объекта **iterable**.

```
lst.extend([7, 8, 9])
```

- **insert(i, x)** : Вставляет элемент **x** на позицию **i**.

```
lst.insert(2, 10)
```

- **remove(x)** : Удаляет первое вхождение элемента **x**.

```
lst.remove(3)
```


- `pop([i])` : Удаляет элемент с позиции `i` (или последний элемент, если индекс не указан) и возвращает его.

```
popped = lst.pop(2)
```

- `clear()` : Удаляет все элементы из списка.

```
lst.clear()
```

2. Методы для работы с содержимым списка:

- `index(x[, start[, end]])` : Возвращает индекс первого вхождения элемента `x` (или по указанному диапазону).

```
idx = lst.index(5)
```

- `count(x)` : Возвращает количество вхождений элемента `x`.

```
occurrences = lst.count(2)
```

3. Методы для сортировки списка:

- `sort(key=None, reverse=False)` : Сортирует элементы списка. Опциональные аргументы `key` и `reverse` позволяют задать функцию для сравнения и сортировку в обратном порядке соответственно.

```
lst.sort()
```

- `reverse()` : Изменяет порядок элементов на обратный.

```
lst.reverse()
```

4. Дополнительные методы:

- `copy()` : Возвращает копию списка.

```
new_lst = lst.copy()
```

- `clear()` : Удаляет все элементы из списка.

```
lst.clear()
```

Кортежи

Основные операции

1. Создание кортежа:

- С помощью круглых скобок:

```
tup = (1, 2, 3, 4, 5)
```

- Без скобок (через запятые):

```
tup = 1, 2, 3, 4, 5
```

2. Доступ к элементам кортежа:

- По индексу:

```
first_element = tup[0] # 1
```

- Срезы (slice):

```
subset = tup[1:4] # (2, 3, 4)
```

3. Особенности кортежей:

- **Неизменяемость:** Кортежи являются неизменяемыми, что означает, что после создания кортежа нельзя изменить его содержимое. Однако, если кортеж содержит изменяемые объекты (например, списки), то их можно изменять.

```
tup[0] = 10 # Ошибка: кортежи не поддерживают присваивание элементам
```

Методы

- `count(value)` : Возвращает количество вхождений `value` в
- `index(value)` : Возвращает индекс первого вхождения `value` .

кортеже.

```
occurrences = tup.count(2)
# Если 2 встречается в кортеже,
# вернет 1
```

```
idx = tup.index(3)
# Вернет индекс первого вхожде
# ния числа 3
```

Преобразование кортежей:

- `tuple(iterable)` : Преобразует итерируемый объект в кортеж.

```
lst = [1, 2, 3]
tup_from_list = tuple(lst) # (1, 2, 3)
```

Множества

Основные операции

1. Создание множества:

- С помощью фигурных скобок:
- С помощью функции `set()` :

```
s = {1, 2, 3, 4, 5}
```

```
s = set([1, 2, 3, 4, 5])
```

2. Основные характеристики множеств:

- **Уникальность элементов:** Множество содержит только уникальные элементы. Повторяющиеся элементы автоматически удаляются.

```
s = {1, 2, 2, 3, 3, 3}
print(s) # Вывод: {1, 2, 3}
```

- **Неупорядоченность:** Элементы множества не имеют определенного порядка, поэтому нельзя получить доступ к элементам по индексу.

Методы

- **add(element)** : Добавляет элемент в множество. Если элемент уже присутствует, он не добавляется.

```
s.add(6)
```

- **remove(element)** : Удаляет элемент из множества. Если элемент отсутствует, вызывает ошибку.

```
s.remove(3)
```

- **discard(element)** : Удаляет элемент из множества. Если элемент отсутствует, ничего не происходит (без ошибки).

```
s.discard(4)
```

- **pop()** : Удаляет и возвращает произвольный элемент из множества. Если множество пусто, вызывает ошибку.

```
popped_element = s.pop()
```

- **clear()** : Удаляет все элементы из множества.

```
s.clear()
```

Операции с множествами:

- **Объединение (|)**: Возвращает множество, содержащее все уникальные элементы из обоих множеств.

```
s1 = {1, 2, 3}
s2 = {3, 4, 5}
union_set = s1 | s2  # {1, 2, 3, 4, 5}
```

- **Пересечение (&)**: Возвращает множество, содержащее только те элементы, которые присутствуют в обоих множествах.

```
intersection_set = s1 & s2  # {3}
```

- **Разность (-)**: Возвращает множество, содержащее все

- **Симметричная разность (^)**: Возвращает множество, содержащее элементы, которые

элементы из первого множества,
которые отсутствуют во втором.

```
difference_set = s1 - s2  # {1, 2}
```

присутствуют только в одном из
множеств.

```
symmetric_difference_set = s1 ^ s2  
# {1, 2, 4, 5}
```

Словари

Основные операции

1. Создание словаря:

- С помощью фигурных скобок:

```
d = {"key1": "value1", "key2": "value2"}
```

- С помощью функции `dict()`:

```
d = dict(key1="value1", key2="value2")
```

- Из списка кортежей

```
d = dict([  
    ('key1', 'value1'),  
    ('key2', 'value2')  
])
```

2. Основные характеристики словарей:

- **Ключи:** Словари используются для хранения пар "ключ-значение". Ключи должны быть уникальными и неизменяемыми (например, числа, строки, кортежи).
- **Значения:** Значения могут быть любого типа и могут повторяться.

3. Доступ к элементам словаря:

- **По ключу:**

```
value = d["key1"] # Возвращает "value1"
```

- **С помощью метода `get()` :**
Возвращает значение для заданного ключа или `None` , если ключ отсутствует.

```
value = d.get("key1", default_value)
```

4. Добавление и изменение элементов:

- **Добавление нового элемента:**

```
d["key3"] = "value3"
```

- **Обновление значения:**

```
d["key1"] = "new_value1"
```

5. Удаление элементов:

- **С помощью оператора `del` :**

```
del d["key1"]
```

- **С помощью метода `pop()` :**
Удаляет элемент по ключу и возвращает его значение. Если ключ отсутствует, можно указать значение по умолчанию.

```
value = d.pop("key1", default_value)
```

- **Удаление всех элементов:**

```
d.clear()
```

Методы

- **`keys()` :** Возвращает представление всех ключей в словаре.
- **`values()` :** Возвращает представление всех значений в словаре.

```
keys = d.keys()
```

- `items()` : Возвращает представление всех пар (ключ, значение) в словаре.

```
items = d.items()
```

```
values = d.values()
```

- `update(other_dict)` : Объединяет словарь с другим словарем или итерируемым объектом.

```
d.update({"key4": "value4"})
```

Функции

Основное

Объявление функции:

```
def greet(name):  
    """Возвращает приветствие  
    для указанного имени."""  
    return f"Привет, {name}!"
```

Функция `greet` принимает имя и возвращает строку приветствия.

Параметры функции:

```
def add(a, b=0):  
    return a + b
```

Функция `add` складывает два числа. Второй аргумент имеет значение по умолчанию равное 0.

Аргументы функции (`*args` и `**kwargs`):

```
def example_args_kwargs(arg1, arg2, *args, **kwargs):  
    print(arg1, arg2)  
    print(args)  
    print(kwargs)
```

Функция принимает два позиционных аргумента и собирает все остальные позиционные аргументы в кортеж `args` и именованные аргументы в словарь `kwargs`.

Лямбда-функции

```
double = lambda x: x * 2
```

Описание: Лямбда-функция `double` умножает переданный аргумент на 2.

ООП

Классы

- **Класс** - это шаблон или определение, которое описывает состояние (атрибуты) и поведение (методы) объектов.
- `__init__` - это конструктор класса, который вызывается при создании нового объекта.
- **Объект** - экземпляр класса. Это конкретный экземпляр класса, который имеет свое уникальное состояние.
- Атрибуты - это переменные, связанные с объектом. Они хранят данные, которые принадлежат объекту.

```
class Person:  
    def __init__(self, name):  
        self.name = name
```

- Методы - это функции, связанные с классом, которые определяют поведение объектов.

```
class Dog:  
    def bark(self):  
        return "Woof!"
```

Описание: `bark` - это метод класса `Dog`, который определяет поведение лаяния для объектов этого класса.

```
class Circle:  
    pi = 3.14159
```

- Создание и вызов объектов:

```
person1 = Person("Alice")  
print(person1.name) # Вывод: Alice  
  
dog = Dog()  
print(dog.bark()) # Вывод: Woof!
```

Описание: С помощью конструктора `__init__` вы создаете новые объекты, и

затем используете их атрибуты и методы.

Наследование

Наследование — это механизм, позволяющий одному классу (подклассу) получить атрибуты и методы другого класса (суперкласса). С помощью наследования можно создавать иерархии классов, где одни классы расширяют функциональность других, используя уже существующие атрибуты и методы.

Описание кода: Класс `Dog` наследуется от класса `Animal`. Метод `make_sound` переопределен в классе `Dog`.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
        return "Woof!"
```

Инкапсуляция

Инкапсуляция — это принцип, который позволяет скрыть внутреннее состояние объекта от прямого доступа и изменения извне. Это делается с целью упрощения работы с объектами и защиты их данных от неправильного использования.

Описание: Атрибут `__speed` является приватным, и к нему можно получить доступ только через методы `set_speed` и `get_speed`.

```
class Car:
    def __init__(self):
        self.__speed = 0

    def set_speed(self, speed):
        self.__speed = speed

    def get_speed(self):
        return self.__speed
```

Полиморфизм

Полиморфизм — возможность объектов разных классов использовать одинаковые интерфейсы или методы. С другими словами, объекты могут быть обработаны как экземпляры их базовых классов, даже если они на самом деле являются экземплярами производных классов.

Описание: Функция `move` принимает объекты различных классов и вызывает соответствующие методы (полиморфизм).

```
class Bird:
    def fly(self):
        return "Flying!"

class Fish:
    def swim(self):
        return "Swimming!"

def move(animal):
    if hasattr(animal, 'fly'):
        return animal.fly()
    elif hasattr(animal, 'swim'):
        return animal.swim()
```

Композиция

Композиция — это принцип, при котором объект одного класса включает в себя объекты других классов в качестве своих частей. Это отличается от наследования, где один класс наследует поведение или атрибуты от другого.

Описание: Класс `Car` содержит объект `Engine`, что представляет собой пример композиции.

```
class Engine:
    def start(self):
        return "Engine started!"

class Car:
    def __init__(self):
        self.engine = Engine()

    def start(self):
        return self.engine.start()
```

Исключения

Основное

Что такое исключение?

Исключение — это событие, которое прерывает нормальное выполнение программы. Когда исключение возникает, выполнение кода переходит к соответствующему блоку обработки исключений.

Базовая структура исключений

```
try:
    # код, который может вызвать исключение
except Имя_Исключения as переменная:
    # код обработки исключения
else:
    # код, который выполняется, если исключение не было вызвано
finally:
    # код, который выполняется всегда, независимо от того,
    # возникло исключение или нет
```

Некоторые стандартные исключения

- **ZeroDivisionError** : Деление на ноль.

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Ошибка: {e}")

# Ошибка: division by zero
```

- **TypeError** : Операция применена к объекту несоответствующего типа.

```
try:
    result = "10" + 5
except TypeError as e:
    print(f"Ошибка: {e}")

# Ошибка: can only concatenate str (not "int") to str
```

- **ValueError** : Функция получает аргумент правильного типа, но неправильного значения.

- **IndexError** : Индекс выходит за пределы последовательности.

```
my_list = [1, 2, 3]
try:
```

```
try:
    result = int("ten")
except ValueError as e:
    print(f"Ошибка: {e}")

# Ошибка: invalid literal for int() with base 10: 'ten'
```

```
print(my_list[10])
except IndexError as e:
    print(f"Ошибка: {e}")

# Ошибка: list index out of range
```

Генерация стандартных исключений

Простейший способ генерации исключения — использование команды `raise` без аргументов. Это создаст экземпляр класса `ValueError`.

```
raise ValueError
```

Однако чаще всего нам нужно добавить сообщение об ошибке:

```
raise ValueError("Неправильное значение!")
```

Генерация собственных исключений

Создание собственного исключения может быть полезным для более точного и ясного определения ошибок в вашем коде.

```
class MyError(Exception):
    pass
```

Теперь вы можете генерировать это исключение так же, как и любое другое:

```
raise MyError("Это мое собственное исключение!")
```

Дополнительные аргументы и исключения

В зависимости от нужд, вы можете передавать дополнительные аргументы при генерации исключения:

```
try:
    x = int(input("Введите число: "))
    if x < 0:
        raise ValueError("Число не может быть отрицательным!", x)
except ValueError as e:
    print(f"Ошибка: {e}")
    # Дополнительная информация из исключения
    print(f"Дополнительная информация: {e.args[1]}")
```

Обработка исключений без указания конкретного типа

Вы можете и не указывать конкретный тип ошибки.

Примечание: Хотя такой подход может быть удобным, его следует использовать осторожно, чтобы не перехватывать неожиданные исключения.

```
try:
    # какой-то код
except:
    # код обработки исключения для всех типов исключений
```

Встроенные функции

Основные

- `print()` - выводит информацию на экран.
- `len()` - возвращает длину объекта (количество элементов).

```
print("Привет, мир!")
```

```
length = len([1, 2, 3, 4, 5])
```

- `type()` - возвращает тип объекта.
- `input()` - считывает ввод с клавиатуры.

```
t = type("строка")
```

```
name = input("Введите ваше имя: ")
```

- `abs()`

Возвращает абсолютное значение числа.

```
value = abs(-5)
```

- `round()`

Округляет число до заданного числа десятичных знаков (по умолчанию 0).

```
rounded = round(3.14159, 2)  
# Вывод: 3.14
```

- `range()`

Возвращает последовательность чисел от начального до конечного значения с заданным шагом.

```
nums = list(range(5))  
# [0, 1, 2, 3, 4]  
even_nums = list(range(0, 10, 2))  
# [0, 2, 4, 6, 8]
```

Функции для работы с последовательностями

- `sorted()` - возвращает новый отсортированный список.

```
sorted_list = sorted([3, 1, 2])
```

- `sum()` - возвращает сумму всех элементов последовательности.

```
total = sum([1, 2, 3])
```

- `max()` и `min()` - возвращают максимальное и минимальное значение из последовательности соответственно.

```
max_val = max([1, 2, 3])  
min_val = min([1, 2, 3])
```

- `zip()`

Объединяет элементы из нескольких итерируемых объектов.

```
list1 = [1, 2, 3]  
list2 = ['a', 'b', 'c']
```

- `enumerate()`

Итерируется по объекту и возвращает индекс и значение.

```
for index, value in enumerate(['a', 'b', 'c']):  
    print(index, value)
```

- `any()` и `all()`

`any()` возвращает `True`, если хотя бы один элемент истинный. `all()` возвращает `True`, если все элементы истинные.

```
print(any([True, False, True]))  
# True  
print(all([True, True, True]))  
# True
```

```
zipped = list(zip(list1, list2)) # [(1, 'a'), (2, 'b'), (3, 'c')]
```

- `reversed()`

Возвращает обратный итератор.

```
reversed_list = list(reversed([1, 2, 3]))  
# [3, 2, 1]
```

Декораторы

Основное

Декораторы позволяют добавлять новую функциональность существующим функциям без изменения их кода.

Основа декоратора

```
def декоратор(func):  
    def обертка():  
        # код перед вызовом функции  
        func()  
        # код после вызова функции
```

```
return обертка
```

```
@декоратор
def привет():
    print("Привет!")
```

Реальный пример декоратора

Декоратор для дебага может выводить информацию о том, какая функция была вызвана и с какими аргументами. Давайте напишем такой декоратор:

```
def debug_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Вызов функции {func.__name__} с аргументами {args} и ключевыми аргументами {kwargs}")
        result = func(*args, **kwargs)
        print(f"Функция {func.__name__} завершила работу, возвращая {result}")
    return result
    return wrapper
```

Теперь, чтобы использовать этот декоратор для любой функции, просто примените его с помощью символа `@` перед определением функции:

```
@debug_decorator
def add(a, b):
    return a + b

result = add(3, 4)
print(result)
```

При вызове функции `add` с аргументами `3` и `4` вы увидите следующий вывод:

```
Вызов функции add с аргументами (3, 4) и ключевыми аргументами {}
Функция add завершила работу, возвращая 7
```


Этот декоратор помогает отслеживать, какие функции вызываются и с какими аргументами, что может быть полезно при отладке и разработке.

Генераторы

Основные

Генераторы позволяют создавать итераторы простым и чистым способом.

Функциональный генератор

```
def my_generator():
    for i in range(5):
        yield i

gen = my_generator()
```

Генераторы списков (List comprehensions)

```
lst = [x for x in range(10) if x % 2 == 0]
```

Генераторы кортежей

```
tup = (x for x in range(10) if x % 2 == 0)
```

Рекурсия

Рекурсия — это процесс, при котором функция вызывает сама себя.

Основные элементы рекурсии:

1. **Базовый случай (Base Case):** Это условие, при котором рекурсивные вызовы прекращаются. Без базового случая рекурсия может привести к бесконечному циклу.

Пример рекурсивной функции (факториал):

```
def factorial(n):
    if n == 0:
        return 1 # базовый случай
    else:
```

2. Рекурсивный случай (Recursive Case):

Это часть функции, где функция вызывает саму себя с измененными аргументами.

```
return n * factorial(n - 1) # рекурсивный случай
```

Основные принципы работы с рекурсией:

1. **Убедитесь, что у вас есть базовый случай:** Это гарантирует, что рекурсия завершится.
2. **Обеспечьте прогресс:** В рекурсивном случае аргументы функции должны изменяться таким образом, чтобы приближаться к базовому случаю.
3. **Будьте осторожны с глубиной рекурсии:** Слишком глубокая рекурсия может привести к переполнению стека вызовов.
4. **Оптимизация:** В некоторых случаях рекурсия может быть не самым эффективным решением, поэтому стоит рассмотреть возможность использования итерации или других методов оптимизации.
5. **Избегайте бесконечных рекурсивных циклов:** Убедитесь, что ваш базовый случай действительно будет достигнут.